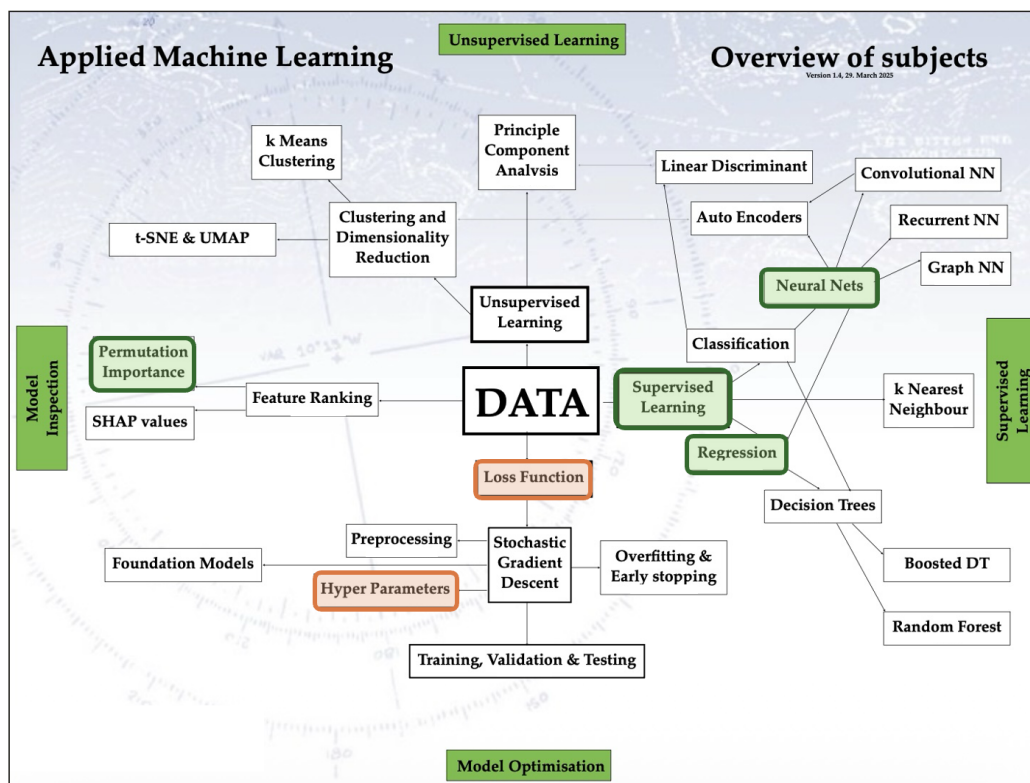


Modul Pendul intro til Machine Learning

STEMAcademyML

Januar 2026

Følgende er en introduktion til Machine Learning (ML) som kan bruges i fysikundervisningen. Den er ment som en kort intro der giver en fornemmelse for hvordan ML virker, og hvordan det kan bruges i videnskab, men går ikke ned i tekniske detaljer. Det kræver ingen kendskab til ML i forvejen. Hvis man gerne vil dykke dybere ned i hvordan en ML algoritme teknisk og matematisk fungerer, er der en mere dybdegående forklaring i forløbene ”standard” på siden.



Der er mange grene af Machine Learning som vist i figuren ovenfor. I denne introduktion vil vi berøre nogle enkelte dele. Fokus vil være på det vi kalder ”regression”, som betyder at man træner en model til at gætte en værdi, modsat ”Classification”, der kan bruges til at forudsige ”ja/nej” eller ”sandt/falsk”. Måden modellen bliver opbygget på er ved et ”Neuralt netværk”, som er det de fleste har hørt om, når man snakker machine learning. Derudover vil vi bruge under ”feature ranking” det vi kalder ”permutation importance”, som kan vise os hvilke variable i vores data der har størst indflydelse, og som er en stor hjælp når vi skal fortolke vores data. Vi vil også meget overfladisk (de orange) berøre ”Loss function”, som er den måde algoritmen lærer på, og ”hyperparameter”, som kan justeres for at optimere algoritmen.

Modulet er bygget op i 3 dele. De 3 moduler er lavet sådan at de kan udføres på 3 moduler af 90 min i en gymnasieklasse, hvor et af modulerne bruges i laboratoriet. Den første del giver en introduktion til hvordan en ML algoritme fungerer, hvordan man træner den og hvordan man evaluerer den. I slutningen af den første del, er der opgaver i hvordan man selv træner en algoritme med data. Det andet modul giver mulighed for at tage data i laboratoriet. Her er en forsøgsvejledning, og forskellige overvejelser man skal gøre sig med forsøgene. Den tredje del går ud på at teste den model man trænede i første del med det data der er indsamlet i lab. Er det muligt for algoritmen at komme frem til det samme resultat som blev målt i lab? Hvordan klarer algoritmen sig i forhold til de teoretiske ligninger? Der er også forskellige spørgsmål til overvejelse om hvornår det er nytting at bruge en algoritme, og hvilke svagheder algoritmen har.

Introduktion

Som fysikere, er vi interesserede i at finde ud af hvordan verden hænger sammen. Det gør vi ofte ved at prøve os frem med eksperimenter og se om vi kan finde en regel eller en ligning, der passer til data.

I dette modul bruger vi data fra et fysik-eksperiment og giver dem til en **machine learning**-model. Vi vil blandt andet undersøge:

- Kan modellen lære at forudsige resultatet ud fra målingerne?
- Kan modellen fortælle os, hvilke parametre der betyder mest?
- Kan modellen klarer sig ligeså godt, eller måske bedre end de teoretiske ligninger?

Forestil dig at du skal prøve at beskrive et pendul, men du kender ikke en ligning for den. Du kan se, at det svinger på en regelmæssig måde, og at tiden ændrer sig når du justerer nogle af parameterne. Men hvordan I alverden skal den ligning se ud?

I stedet for at gætte en formel fra starten, kan vi lade en computer finde mønstre i data. *Det* er idéen bag Machine Learning.

Pendul

Rent teoretisk har vi en meget smuk og enkel ligning for svingningstiden (også kaldet perioden) på et pendul.

$$T = 2\pi\sqrt{\frac{L}{g}} \quad (1)$$

Her er T perioden (sekunder), L snorens længde (meter) og g tyngdeaccelerationen (ca. $9,82 \text{ m/s}^2$). Vi ser at svingningstiden faktisk kun afhænger af snorens længde, resten er konstanter. Men, i udledningen af denne ligning bliver der lavet en del antagelser, blandt andet at:

- snoren er masseløs (massemidtpunkt er i midten af loddet)
- vinklen er lille (noget vi ofte gør i teoretiske udledninger kaldet small angle approximation)
- luftmodstand er ubetydelig

Til eftertanke...

- Hvorfor kan vi lave disse antagelser? Og holder de i den virkelige verden?
- I hvilke tilfælde ville ligningen måske ikke længere passe?
- I den virkelige verden, hvilke effekter tror I vil påvirke svingningstiden? Prøv at tegne et kraftdiagram for pendulet.

1 Modul 1: Neurale netværk

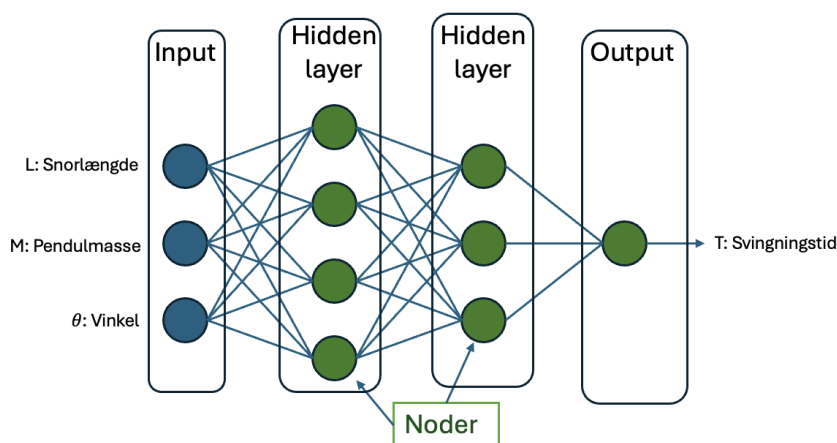
Machine learning kan være et stærkt værktøj i fysik, fordi det kan finde mønstre i data. Det er især nyttigt, når der er for mange parametre og variable til at vi kan se en klar sammenhæng. En algoritme behøver ikke kende teorien på forhånd, den kan lære sammenhængen direkte ud fra data.

En meget brugt type model er et **neuralt netværk (NN)**. Navnet kommer fra, at opbygningen minder om måden neuroner i hjernen sender signaler på. Et neuralt netværk består af:

- et **input layer** (det vi måler, fx snorlængde)
- et **output layer** (det vi vil forudsige, her perioden T)
- et eller flere **hidden layers** (mellemregninger)

I hvert lag er der **noder** som har **vægte**, og som bestemmer hvor meget hvert input tæller med.

Et typisk netværk kan se sådan ud:



Figur 1: Eksempel på et Neuralt Netværk med Input, Output og 2 Hiddel Layers.

Lad os antage at vi ikke kender ligning 1, men vi ved at snorlængden L er vigtig. Til at state med, kunne vi gætte på en lineær sammenhæng

$$a \cdot \text{snorlængde} + b = T \quad (2)$$

Og så skal vi bare finde ud af hvilke værdier for a og b der kommer tættest på det rigtige resultat. Vi kunne prøve at starte med tilfældige værdier for a og b , og se hvor tæt vi kommer på resultatet, og så justere dem lidt ad gangen indtil vi kommer tæt på.

Til eftertanke...

Tror du en ret linje som ligning 2 kan passe godt til en sammenhæng, der egentlig er en kvadratrods (ligning 1)? Hvad forventer du, at der sker, hvis vi prøver? Kunne en lineær ligning beskrive nogle af områder af data? (tænk over hvordan ligning 1 ser ud)

Det er det samme som en ML algoritme gør. Men i stedet for kun at have en ligning, laver den mange ligninger og kombinerer resultaterne. Det sker ved at vi introducerer **hidden layers**. Et hidden layer giver modellen flere “mellemregninger”. Forestil dig, at netværket består af mange noder, som hver laver en lille beregning på inputtet. Forskellige noder kan “tage højde for” forskellige aspekter: nogle reagerer mere, når snoren er lang, andre når den er kort, nogle bemærker små ændringer, og nogle ser kombinationer af længde og vinkel.

Når vi kombinerer alle disse små bidrag gennem lagene, kan modellen til sidst forudsige perioden på en måde, som én enkelt ligning ikke kunne. På den måde kan netværket automatisk lære hvilke faktorer der er vigtigst for hvert tilfælde.

Pointen er: **flere lag = flere trin = mere fleksibel model** ¹

Virkeligheden er ofte mindre pæn, og flere ting kan påvirke perioden, fx vinkel, luftmodstand (relateret til størrelse/tværsnit) og masseforhold, som ligning 1 ikke tager hensyn til. Men siden vores algoritme er ligeglad med ligningen, kan vi bede den om at lære sammenhængen med flere variable.

Med flere input (og evt. flere hidden layers) kan modellen lære mere komplekse sammenhænge og lave bedre forudsigelser.

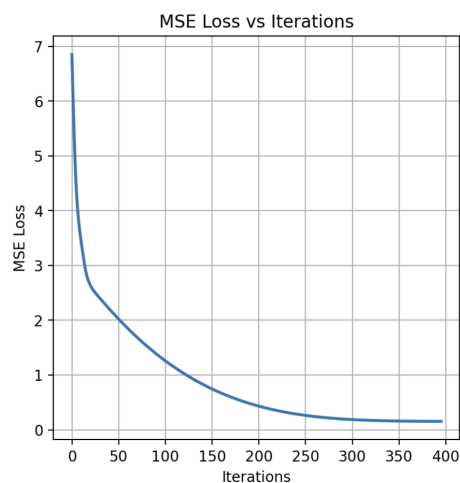
1.1 Hvordan lærer modellen? (træning)

Når vi træner vores model, har vi en masse data, som den skal lære af. I det her tilfælde giver vi den en masse data fra eksperimenter hvor vi har målt forskellige parameter, som snorlængde, vinkel, masse af pendul, og svingningstiden. Når den har en masse eksempler hvor den kender svaret (svingningstiden), kan den lære hvad sammenhænge er.

Når vi træner modellen, gør den sådan her:

1. Den starter med tilfældige vægte (tilfældige $a, b, c, \dots$ i ligningerne i noderne)
2. Den laver en forudsigelse $T_{\text{forudsagt}}$ ved at køre gennem hele netværket
3. Den sammenligner med den rigtige værdi $T_{\text{målt}}$ fra den data den træner på
4. Den justerer vægtene en lille smule, så fejl (forskellen på $T_{\text{forudsagt}}$ og $T_{\text{målt}}$) bliver mindre næste gang

Fejlen kaldes **loss** (tab). Målet er at gøre loss så lille som muligt. Hver gang den prøver med en kombination af parameter, har vi kørt en **iteration**, eller en "træningsrunde". Jo flere iterations vi kører, jo bedre skulle vores model gerne blive. Men på et tidspunkt vil vi også nå til et punkt hvor algoritmen ikke længere forbedrer sig. Det vil ofte være begrænset af de informationer vi har til rådighed. Ofte vil man bygge sin algoritme sådan at hvis den ikke forbedrer sig længere, så skal den stoppe, så vi ikke bruger unødigt meget computerkraft på det.



Figur 2: Her ses hvordan loss falder jo flere iterations modellen får lov at køre. Jo lavere loss, jo bedre er vores model

Eksempel

Lad os sige vi gjorde det med vores lineære ligning (2). Hvis modellen får $L = 1$ m og den målte periode er $T_{\text{målt}} = 2$ s, prøver den først nogle vægte. Hvis den fx gætter $a = 2$ og $b = 2$, får den $T_{\text{forudsagt}} = 2 \cdot 1 + 2 = 4$ og loss er $|4 - 2| = 2$. Hvis den i stedet gætter $a = 2$ og $b = 1$, får den $T_{\text{forudsagt}} = 2 \cdot 1 + 1 = 3$ og loss er 1, altså en forbedring. Modellen gentager dette for mange datapunkter og justerer gradvist, indtil den finder vægte, der generelt giver lav loss for hele datasættet. Jo flere noder og lag der er i vores netværk, jo flere kombinationer skal den prøve.

For mennesker ville det her hurtigt blive en umulig opgave. Alene i det "simple netværk" i figur 1 har vi 35 parameter. Forestil dig at skulle ændre på 35 forskellige værdier lidt ad gangen indtil du rammer rigtigt. Det ville tage dagevis at finde en god kombination.² Men med computerkraft kan vi gøre det på sekunder.

For at modellen kan lære, skal den have mange eksempler med kendte svar, dvs. en masse data hvor svingningstiden er målt. Modellen bruger disse eksempler til gradvist at justere sine vægte, så

¹Matematikken bag og hvordan de forskellige parameter spiller sammen er en lidt længere forklaring, men hvis man er interesseret, er de forklaret mere i dybden i nogle af de andre moduler på STEM Academy, som for eksempel "Huspriser", eller se videoerne under "ekstra materiale".

²Til sammenligning har ChatGPT 1.8 trillioner parameter! (2025)

dens forudsigelser bliver tættere på de målte værdier. Uden data med kendte svar kan modellen ikke vurdere, om dens gæt er korrekte, og den kan derfor ikke lære. Det er også derfor data er så værdifuldt: Jo flere og bedre målinger vi har med "svar", desto bedre kan modellen lære mønstre og lave forudsigelser på ny data.

Til eftertanke...

Hvorfor findes der så mange 'gratis' apps hvor man skal oprette en konto og give sine oplysninger?

1.2 Hvilke parameter er vigtige? (Permutation Importance)

Når vi har trænet vores model, er det interessant at finde ud af, hvilke inputvariabler den lægger mest vægt på. En metode til at vurdere dette kaldes **Permutation Importance**. Ideen er enkel: Tag én variabel (fx snorlængden L), byt rundt på værdierne tilfældigt og se om modellens præcision falder. Hvis præcisionen falder meget, betyder det, at variabelen er vigtig for forudsigelsen. Hvis præcisionen næsten ikke ændrer sig, betyder det, at variabelen ikke bidrager væsentligt.

Eksempel

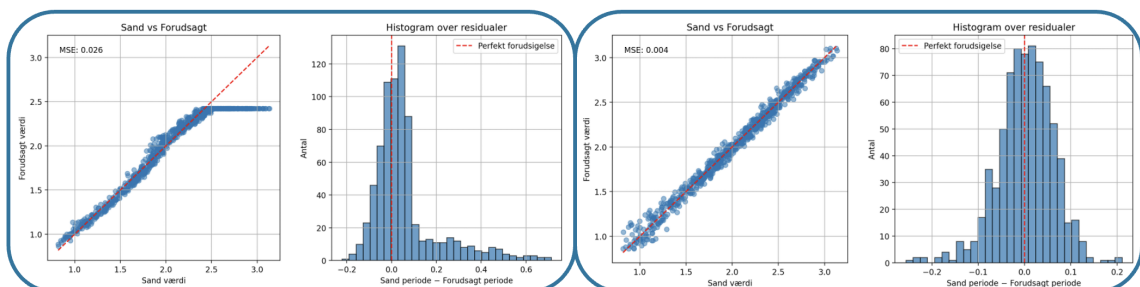
Hvis vi bytter tilfældigt om på vinklen, som for eksempel et forsøg der blev lavet ved en vinkel på 60 grader, men vi i stedet skriver 30 grader, så vinklen ikke er den vi reelt set brugte, og modellens præcision falder markant, betyder det, at vinklen er vigtig for at forudsige svingningstiden. Eller hvis vi bytter rundt på massen, og vi ser at præcisionen eller forudsigelsen er den samme, så ved vi at massen har meget lidt indflydelse.

Permutation Importance giver altså et intuitivt billede af, hvilke faktorer modellen faktisk bruger, og hjælper os med at forstå, hvilke parametre der har størst betydning for resultatet. Så hvis vi for eksempel skulle skrive en ligning for pendulet, så ville vi vide hvilke parameter der var vigtige at få med i en sådan ligning.

1.3 Hvordan vurderer vi hvor godt algoritmen klarer sig?

Når man så har bygget sin algoritme, vil vi gerne finde ud af hvor god den er til at forudsige. Det gør vi ved at give modellen ny data, som den ikke har trænet på, og bede den om at forudsige, og så sammenligne med de rigtige værdier. Typisk deler man datasættet i to dele: en del til **træning**, som vi allerede har berørt, og en mindre del til **test**, som gemmes væk, indtil modellen skal vurderes. På den måde sikrer vi, at modellen ikke bare husker de præcise svar, men faktisk har lært et mønster, der også virker på ny data.

Typisk vil man visualisere performance som vist nedenfor:



Figur 3: Den første model (venstre boks) klarer sig godt i et stort område, men fejler når vi er over 2.5, hvorimod den anden model klarer sig godt på hele området.

Her er vist 2 trænedede modeller. Til venstre i hver boks er plottet forudsagte værdier mod sande værdier. Her vil vi gerne have, at punkterne ligger tæt på linjen $y = x$ (den røde linje), som betyder, at forudsigelsen og den sande værdi er den samme. Tallet "MSE" står for mean squared error og

fortæller hvor langt punkterne ligger fra linjen. Jo tættere tallet er på 0, jo bedre. Til højre er plottet forskellen mellem forudsagte og sande værdier (residualer). Her vil vi gerne have, at punkterne ligger tæt på nul, så forudsigelserne ikke systematisk rammer for højt eller for lavt, og at spredningen (hvor langt punkterne ligger fra 0) er så lav som muligt.

1.4 Opgave: Træn din algoritme

Nu hvor vi har en ide om hvordan en algoritme fungerer, er vi klar til at træne vores egen model. For at træne en model, skal der bruges en stor mængde data, ofte taler vi 10.000 datapunkter og derover, for at algoritmen kan finde meningsfulde mønstre. Heldigvis er der allerede sørget for et stort datasæt til formålet. Det lyder måske af meget, men i fysik hvor vi får data fra for eksempel partikelacceleratoren i CERN, eller data fra andre store maskiner, er det ikke usædvanligt at få data i den størrelse.

I datasættet er der snorlængden, men også en række andre variable, som foreksempel vinklen, tværsnitarealet af loddet, masseforholdet af snoren og loddet, variable som ikke indgår i ligning 1. Men hvis nogen af de her parameter har indflydelse på svingningstiden, vil algoritmen kunne se mønsteret.

Når vi så har en model vi er tilfreds med, kan vi bruge den til at lave forudsigelser. Det vil sige, at hvis vi har en god model, kan vi give den de parameter vi har tænkt os at bruge, som snorlængde, masse af pendul m.f., og så kan den komme med et bud på hvad svingningstiden vil være. Senere vil vi også prøve af om det er rigtigt ved at gå i laboratoriet og lave rigtige forsøg. Herefter går det ud på at se om algoritmen kan forudsige de svingningstider vi måler.

Opgaver

På siden under "Modul Pendul" kan vi øverst vælge datasæt, og inspicere dataen. Vælg det datasæt der indeholder snorlængder op til 1 meter først. Under dataen kan vi vælge hvor meget af dataen algoritmen må få at træne på. Herefter kan vi indstille vores "**hyperparametre**", dvs. de parametre der har indflydelse på hvordan algoritmen virker. I tilfældet her er det antal lag og antallet af noder i hvert lag, der er vores hyperparameter. I plottet kan du se "MSE", og bruge den som en indikator for om din model bliver bedre eller dårligere. Prøv at vælge forskellige kombinationer af lag og noder. Prøv at justere indtil du får et tilfredsstillende resultat. Prøv følgende og overvej:

- Prøv først at give modellen en lille andel af dataen (fx 20%). Hvor mange lag og noder skal der til før dataen begynder at ligge på den røde linje?
- Hvilke parameter er vigtigst ifølge algoritmen (permutation importance)? Stemmer det overens med hvad du ved om pendulet? Er det rimeligt at ligning 1 antager at det kun er snorlængden L der har betydning?
- Prøv at give modellen mere data (andel af data til træning). Kan du se forskel i performance? og i hvor mange lag og noder der skal til?
- Bliver modellen uendeligt bedre jo flere lag og noder du giver den?
- Prøv til sidst at give den det større datasæt med snorlængder op til 2 meter. Er der forskel på de trænede modeller?
- Den bedste model er den der giver den bedste forudsigelse, med færrest parameter (hvorfor?). Hvor synes du det "sweet spot" ligger?

Når du har fundet de "hyperparameter" du synes giver den bedste model, så skriv dem ned, så du kan bygge den samme model igen senere.

2 Modul 2: Dataindsamling (pendulforsøg)

Hvis man ikke har mulighed for at gå i lab, kan man hente ”Eksempel på datasæt til modul pendul” på siden, som er rigtig forsøgsdata, og springe videre til modul 3 og bruge det data i stedet.

Formål

I dette modul skal I samle jeres egen penduldata og bruge dem til at teste, hvor godt modellen kan forudsige perioden T . I undersøger også, hvornår den teoretiske formel passer godt og hvornår den begynder at fejle. Hav derfor i tanke hvilke tilfælde der vil passer godt med den teoretiske ligning (tænk på antagelserne), og hvilke tilfælde der vil falde udenfor. Brug det tilhørende excel ark til at skrive jeres observationer ned. Det er vigtigt at I ikke ændrer i arket, men kun skriver tal ind i felterne med mindre andet er anført, for at det nemt kan uploades til jeres Machine Learning modeller senere.

Materialer

- Stopur (mobil er ok)
- Målebånd eller lineal
- Vinkelmåler
- Ophæng (krog, stativ, loftsbjælke el.lign.) med mindst 2 m til gulvet
- Snor 1: tynd og let (fx fiskesnøre) ca. 2 m
- Snor 2: tungere snor/kæde ca. 2 m
- Lodder i forskellige former og masser (vær kreative).

Fremgangsmåde

1. Vælg en kombination af snor, lod, længde L og vinkel θ .
2. Hæng pendulet op, så der er god afstand til gulvet.
3. Mål og noter data i excel arket
4. Træk loddet ud til den ønskede vinkel og **slip uden at skubbe** (vinklen måles fra lodret).
5. Vælg et fast referencepunkt (fx en tape-streg på væggen).
6. Start stopuret når loddet passerer referencepunktet i én bestemt retning.
7. Skift én ting ad gangen (fx længde, vinkel, lod eller snor) og gentag.

Tip: Det er ofte bedre at måle tiden for 10 svingninger og dividere med 10, end at måle ét sving.

Gode datasæt (så modellen virkelig testes)

Prøv at få data der både er “pæne” og “svære”:

- **Forskellige snorlængder:** vi forventer fra ligning 1 at længden vil have stor betydning.
- **Små og store vinkler** (fx 5° – 10° og $> 50^\circ$): nogle tilfælde vil passe teorien godt, og andre vil afvige - hvilke?
- **Lille vs. stor luftmodstand:** lille lod vs. lod med stort tværsnit.
- **Let vs. tung snor:** fx fiskesnøre vs. kæde.
- **Massefordeling:** tilfælde hvor loddet er tungere end snoren og omvendt.

Til eftertanke...

Hvilke forsøg tror du den teoretiske formel $T = 2\pi\sqrt{L/g}$ forudsiger bedst? Og hvilke forsøg vil den have sværere ved — og hvorfor? (Tænk fx på store vinkler, tung snor og stor luftmodstand.)

3 Modul 3: Man vs Machine

Nu hvor vi har en trænet machine learning algoritme, og en masse data fra vores forsøg, skal vi teste om algoritmen kan forudsige de svingningstider vi har målt. Dernæst vil vi se om vi får det samme hvis vi bruger ligning 1 og kun bruger snorlængden. Og til sidst en lidt mere kompliceret version af ligning 1. Hvilke ting kan spille ind på hvor godt ligningen eller algoritmen klarer sig? Og er der nogen tilfælde hvor den ene klarer sig bedre end den anden om omvendt? Start med at gå ind på websiden samme sted som i modul 1.

Opgave 1

Undersøg din algoritme vs ligningen

1. Træn din model med de hyperparameter du skrev ned sidst, så du har den bedste model.
2. Upload din forsøgsdata og få din trænete algoritme til at forudsige svingningstiden. Hvordan klarer den sig?
3. Tjek boksen af med "simpel ligning" af, og se hvor god ligning (1) er til at forudsige svingningstiden. Hvordan klarer den sig i forhold til algoritmen? Hvilken vil du sige er bedst? Hvorfor?
4. Er der nogen tilfælde hvor algoritmen klarer sig bedre end ligningen og omvendt? Hvilke tilfælde tror du det drejer sig om?

Opgave 2

Undersøg algoritmens svagheder

1. Prøv at vælge datasættet hvor der kun er snorlængder op til 1 meter. Hvordan klarer algoritmen sig nu? Hvordan klarer den sig i forhold til ligningen? Hvad fortæller det om begrænsningerne for en trænet algoritme?

Opgave 3

En mere komplet ligning

Ligning 1 er som sagt en forenklet ligning og bygger på et idealiseret tilfælde. Men vi har også ligninger hvor flere effekter bliver taget med. Tager vi med at snoren ikke er masseløs, at vi ikke bruger small angle approximation og regner luftmodstanden med, vil ligningen se således ud ^a:

$$T = 2\pi \sqrt{\frac{L \cdot f_{snor}}{g}} \cdot \frac{2}{\pi} K(\sin^2[\frac{\theta_0}{2}]) \cdot (1 + \frac{(\frac{\gamma}{\omega_0})^2}{8}) \quad (3)$$

1. Tjek boksen af med "avanceret ligning". Hvordan klarer den sig i forhold til den simple ligning? I forhold til algoritmen?
2. Ovenstående ligning er brugt til at generere data brugt til at træne modellen. Med dette i mente, er det muligt for modellen at klare sig bedre end ligningen? Hvorfor, hvorfor ikke?
3. Kan algoritmen blive uendeligt god hvis bare man træner længe nok/har nok data? Hvilke udfordringer er der? (hint: kig på loss plottet)

^aHvis man er nysgerrig, er formelen uddybet i appendix

Til eftertanke...

- Hvilke fordele og ulemper er der ved at bruge machine learning til at lære fysik?
- Hvilke begrænsninger er der ved machine learning? Kan det erstatte teori? Hvis ikke nu, hvad så med i fremtiden?
- Hvis du ikke kendte ligningen, hvad kunne du så lære om pendulet udelukkende fra Machine Learning resultaterne?

3.1 (Ekstra) Kan vi udlede en ligning ud fra vores algoritme?

Selvom algoritmen ikke kender ligningen, og heller ikke kan give os en, men kun give os sammenhænge og mønstre, så kan vi faktisk bruge den til at give et bud på sammenhængene mellem de enkelte parametre.

Partial dependence plotsne (PDP) under Modul 3 viser hvordan perioden ændrer sig hvis man skruer på én parameter ad gangen. Hver lyseblå linje viser dette for et tilfældigt sample. Den mørkeblå linje er gennemsnittet af ændringerne for alle de lyseblå linjer. Du kan derfor bruge dette til at sammenligne formen af kurverne med formen af kurverne af kendte funktioner. Den lodrette akse svarer til ændringen i perioden.

Eksempel

Du kan se at den mørkeblå linjes kurve ligner grafen for en kvadratisk funktion, $y \approx x^2$. Dette ville tyde på en sammenhæng $T \propto L^2$. Vi kan dog ikke bestemme om der er (eller deres værdi) numeriske faktorer foran variabelen. Det er heller ikke muligt at udføre partial dependence for flere variable ad gangen.

Opgaver

Træn den bedst mulige model, med de bedste parametre du har fundet på 100% af data for datasættet med 'L_measured' i intervallet (0.2m – 2m).

- Minder formen af kurverne for nogen af variablene om kurver af kendte funktioner? Hvilke? Hvordan kan du bruge dette til at give et bud på ligningen af perioden.
- Sammenlign formen af kurverne for variablene med deres form i den simple/komplicerede ligning. Stemmer de overens? (Note: Vinkelafhængigheden i den komplicerede ligning er kvadratisk til ledende orden.)
- Hvad betyder det at formen af grafen for nogle af variablene er helt flade? Sammenlign dette med disse variables betydning i Permutation Importance. Giver det mening?
- Hvad sker der med PDP hvis du træner en 'dårlig' model på lidt data. Hvad betyder det for modellen at PDP viser forkerte sammenhænge?

4 Appendix

4.1 Ligning for pendul

Ligning 1 er som sagt en forenklet ligning og bygger på et idealiseret tilfælde. Men vi har også ligninger der tager højde for ikke ideelle tilfælde med luftmodstand, ikke-masseløs snor m.fl.. Denne ligning kunne se således ud

$$T = T_0 \cdot T_{angle} \cdot T_{drag} \quad (4)$$

hvor T_0 er perioden for et simpelt pendul med en massiv snor, T_{angle} er bidraget der kommer når man tager højde for at vinklen kan være arbitrær (ingen small-angle approximation), T_{drag} er bidraget fra luftmodstanden. De ser således ud

$$T_0 = 2\pi \sqrt{\frac{L \cdot f_{snor}}{g}}, \quad (5)$$

hvor f tager højde for at snoren har masse og er givet ved

$$f_{snor} = \frac{\frac{1}{3}\alpha + (1 - \alpha)}{\frac{1}{2}\alpha + (1 - \alpha)}, \quad (6)$$

$$T_{angle} = \frac{2}{\pi} K(\sin^2[\frac{\theta_0}{2}]), \quad (7)$$

hvor $K(m)$ er det komplette eliptiske integrale af første slags defineret her og

$$T_{drag} = 1 + \frac{(\frac{\gamma}{\omega_0})^2}{8}, \quad (8)$$

med $\omega_0 = \frac{2\pi}{T_0}$ og $\gamma = (\rho_{air} \cdot C_d \cdot A \cdot L)/m_{total}$. C_d er formfaktoren af loddet (antaget cylindrisk) og ρ_{air} er lufttætheden. Ligningen medtager den eksakte vinkelafhængighed. Den tager højde for at jo mere masse der ligger i snoren, det svarer til et simpelt pendul med en kortere snor, effektivt set. Luftmodstanden beregnes ud fra antagelsen om kvadratisk luftmodstand (den 'normale' formel for luftmodstand). Denne er lineariseret under betingelsen om at energien mistet på en svingning er den samme i det ægte tilfælde som i det lineariserede. Herefter regnes en dæmpfaktor baseret på denne antagelse. Formen af udtrykket kommer af en taylorudvikling af svingningstiden for en dæmpet harmonisk oscillator. Detaljerne er mindre vigtige men der er fysisk grundlag for det hele. Pointen er at et pendul er virkelig besværligt at beskrive hvis man tager alle virkelighedens effekter med.

4.2 Data

Til opgaven hører en skabelon-excelfil hvor man kan indtaste forsøgsdata. Denne er kompatibel med hjemmesiden, og kan uploades direkte under Modul 3 for at lave forudsigelser med modellen på eget forsøgsdata. Der findes også en lignende fil *med data*. Denne skal ses som en erstatning for hvis man ikke har mulighed for at tage data selv. De ligger begge i venstre spalte på hjemmesiden under Modul Pendul.

Data indeholder:

- *Pendulets længde*: L-measured [m], målt i meter. Måles fra toppen af snoren til massemidtpunktet af loddet.
- *Andel af masse i snoren*: alpha [1], enhedsløs. Regnes som $\alpha = \frac{m_{snor}}{m_{snor} + m_{lod}}$.
- *Startvinkel*: theta0 [rad], målt i radianer. Måles fra lodret. Du kan omregne fra grader til radianer ved at gange med $\frac{\pi \text{ rad}}{180 \text{ deg}}$.
- *Tværsnitsareal af lod*: crosssection [m²], målt i kvadratmeter. Tværsnittet regnes i bevægelsesretningen af loddet.
- *Total masse af lod og snor*: m_total [kg], målt i kilogram.
- *Periode*: Periode [s], målt i sekunder.